DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

CS23432 – Software Construction

RAJALAKSHMI ENGINEERING COLLEGE
Thandalam, Chennai-602015

Name: Rahul G

Register No: 241901087

Year / Branch / Section: 3rd / CSE-CS/

Semester: V

Academic Year: 2026 - 2027

BONAFIDE CERTIFICATE

Certified that this project report "**NETWORK MONITORING SYSTEM**" is the bonafide work of "**RAHUL G**" who carried out the project work under my supervision.

SIGNATURE

Mrs. Rupmala
Assistant Professor
Dept. of Computer Science and Engineering, (Cyber Security)
Rajalakshmi Engineering College
Chennai

Submitted for the Practical Examination held on _____

EXAMINER

INDEX

S.No.	Date	Title
1.	16/07/26	Azure Devops Environment Setup.
2.	23/07/26	Azure Devops Project Setup and User Story Management.
3.	30/07/26	Setting Up Epics, Features, And User Stories for Project Planning.
4.	06/08/26	Sprint Planning.
5.	13/08/26	Poker Estimation.
6.	03/09/26	Designing Class and Sequence Diagrams for Project Architecture.
7.	10/09/26	Designing Architectural and ER Diagrams for Project Structure.
8.	17/09/26	Testing – Test Plans and Test Cases.
9.	24/09/26	Load Testing and Pipelines.
10.	08/10/26	GitHub: Project Structure & Naming Conventions.

EXP NO: 1	AZURE DEVOPS ENVIRONMENT SETUP

Aim:

To set up and access the Azure DevOps environment by creating an organization through the Azure portal.

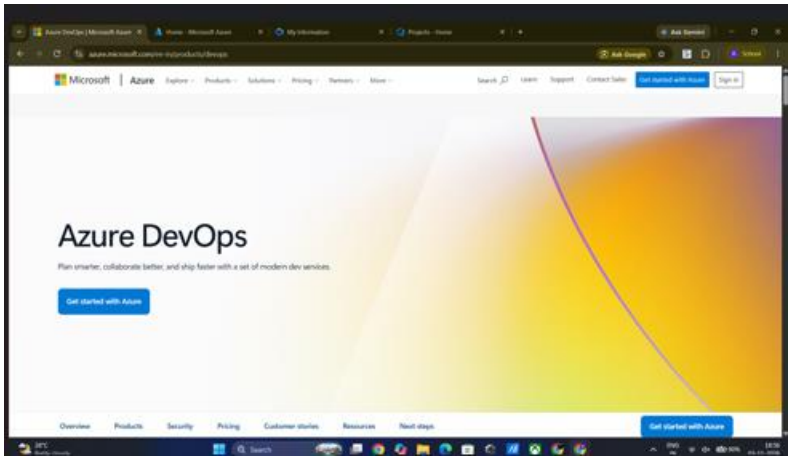
INSTALLATION

1. Open your web browser and go to the Azure website: https://azure.microsoft.com/en-us/get-started/azure-portal.

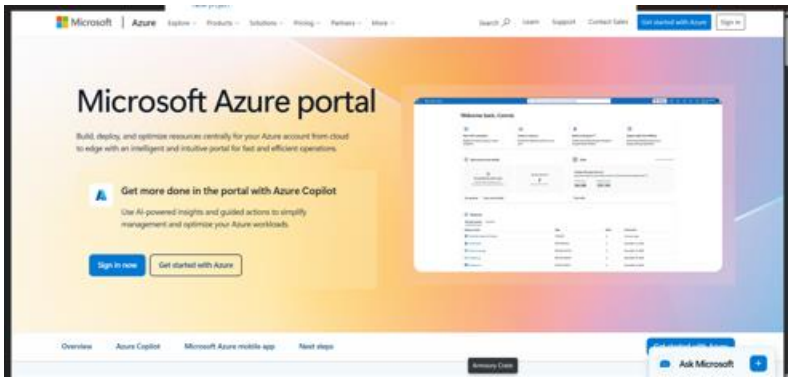
Sign in using your Microsoft account credentials.

If you don't have a Microsoft account, you can create one here: https://signup.live.com/?lic=1

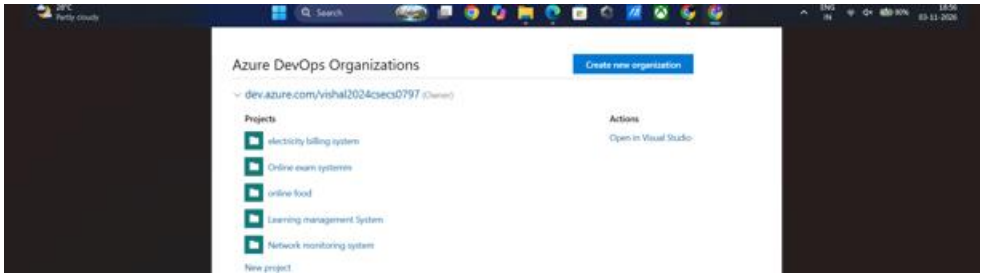
2. Azure home page



3. Open DevOps environment in the Azure platform by typing ***Azure DevOps Organizations*** in the search bar.



4. Click on the ***My Azure DevOps Organization*** link and create an organization and you should be taken to the Azure DevOps Organization Home page.



Result:

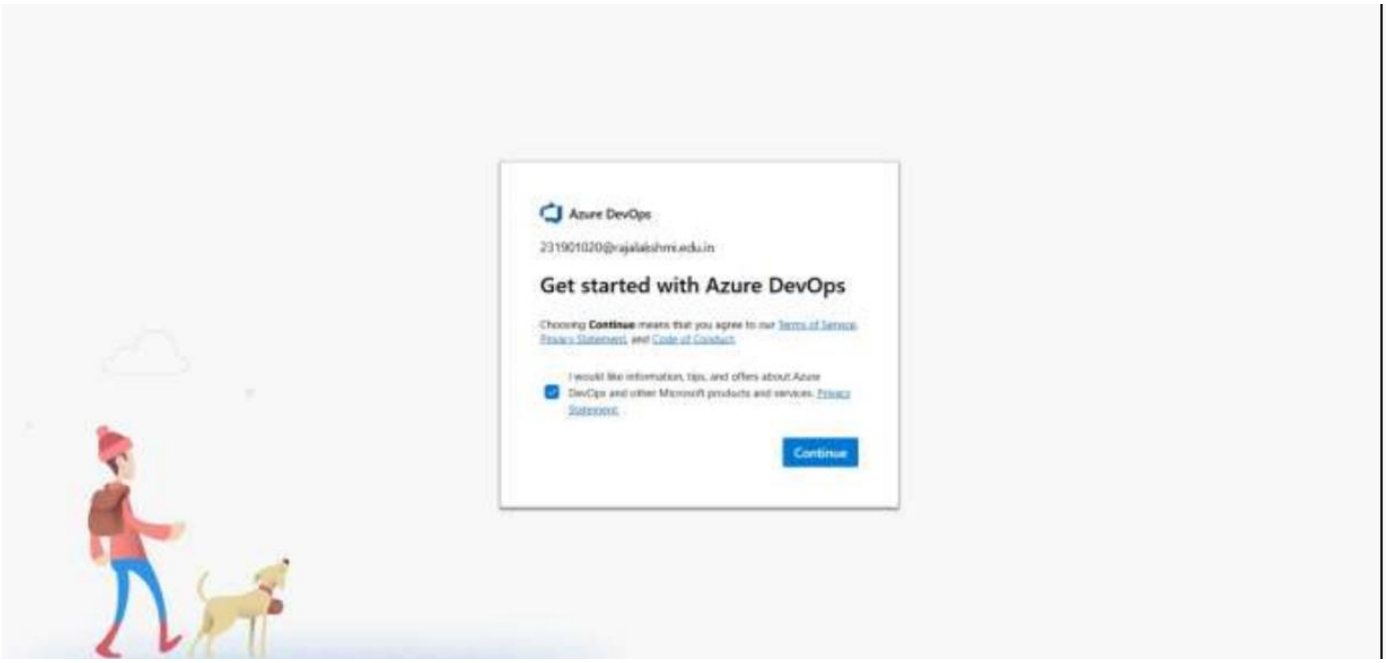
Successfully accessed the Azure DevOps environment and created a new organization through the Azure portal.

EXP NO: 2	AZURE DEVOPS PROJECT SETUP AND USER STORY MANAGEMENT

Aim:

To set up an Azure DevOps project for efficient collaboration and agile work management.

1. Create An Azure Account



2. Create the First Project in Your Organization

a. After the organization is set up, you'll need to create your first **project**. This is where you'll begin to manage code, pipelines, work items, and more.

b. On the organization's **Home page**, click on the **New Project** button.

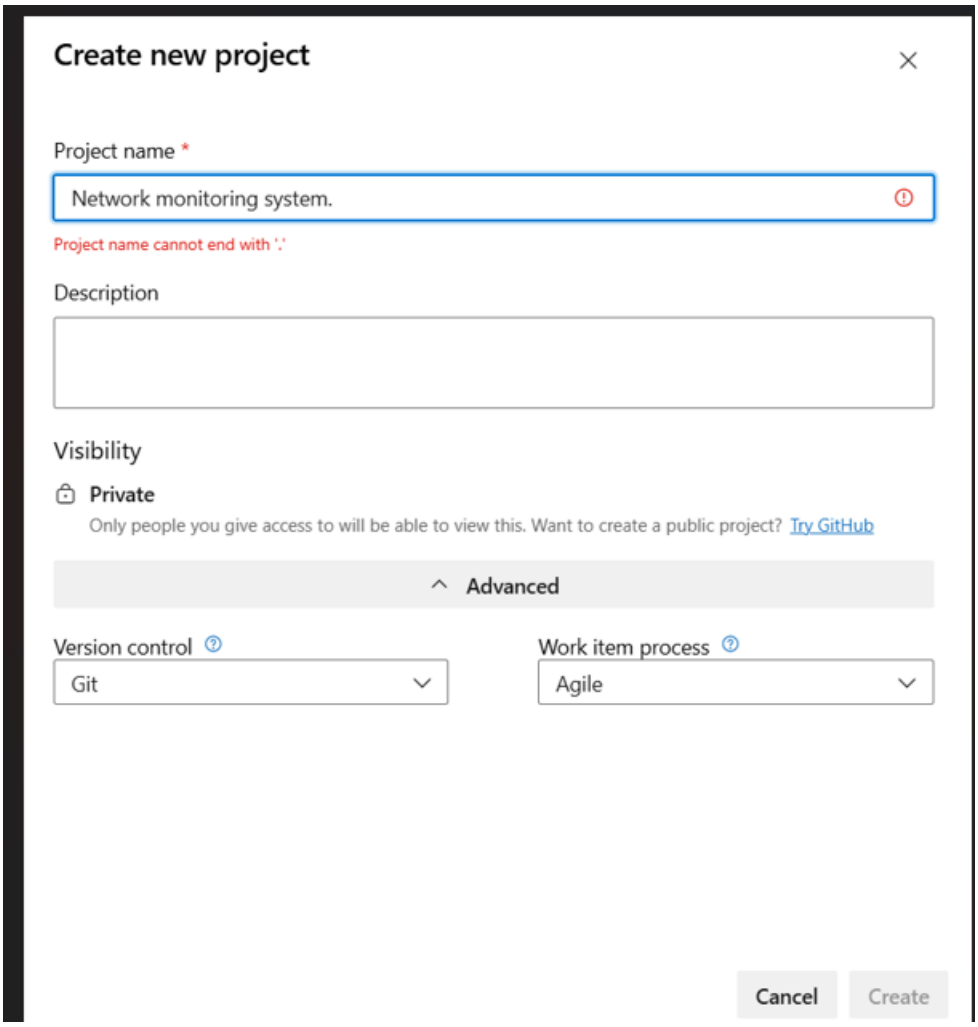
c. Enter the project name, description, and visibility options:

Name: Choose a name for the project (e.g., LMS).

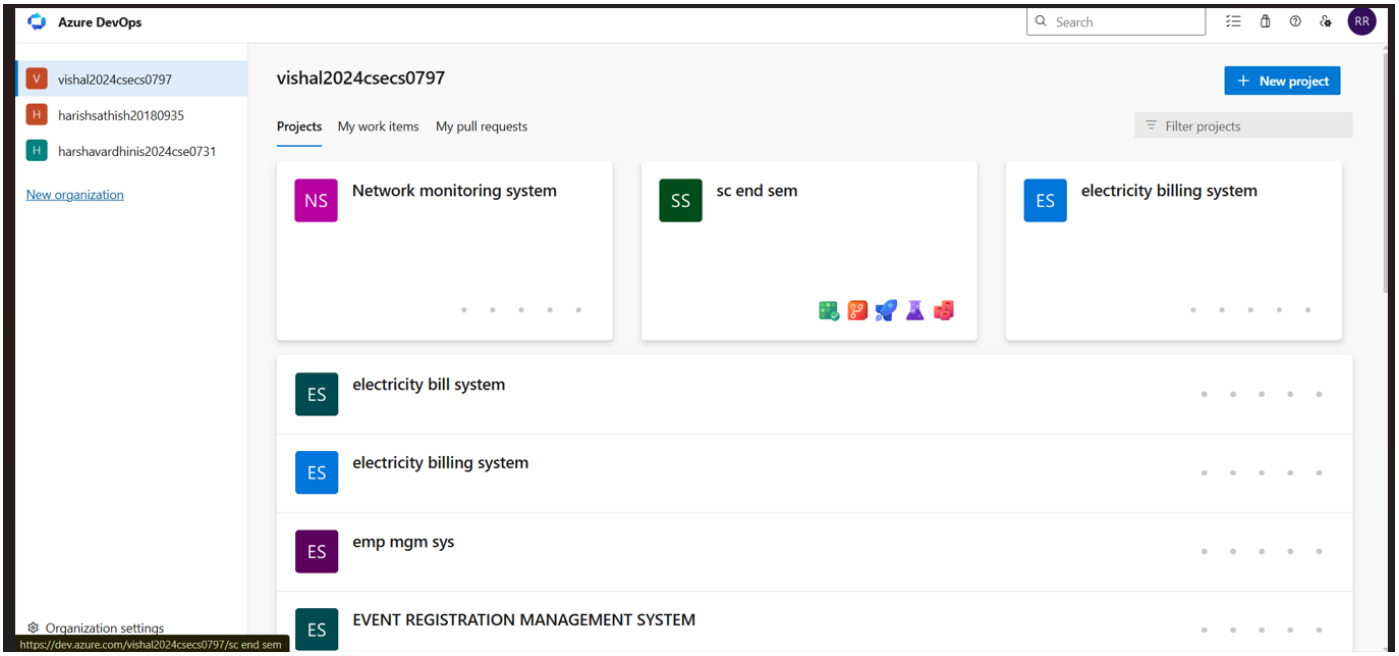
Description: Optionally, add a description to provide more context about the project.

Visibility: Choose whether you want the project to be **Private** (accessible only to those invited) or **Public** (accessible to anyone).

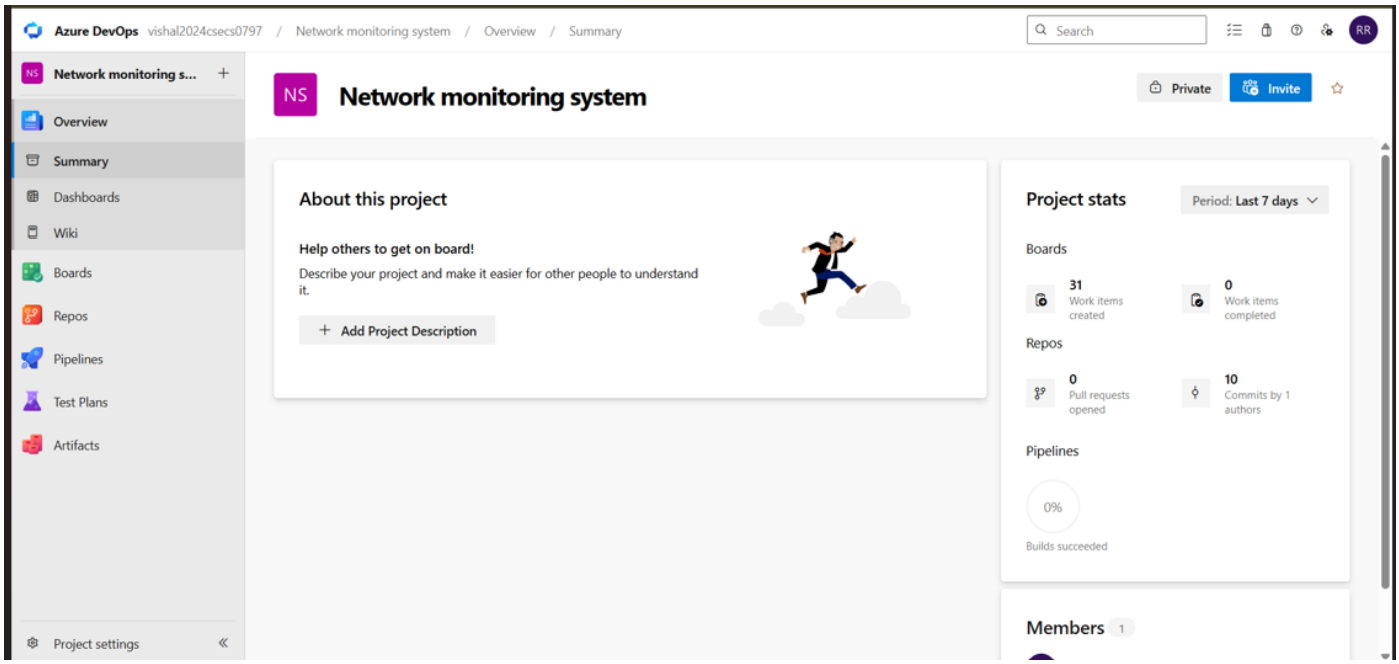
d. Once you've filled out the details, click **Create** to set up your first project.



3. Once logged in, ensure you are in the correct organization. If you're part of multiple organizations, you can switch between them from the top left corner (next to your user profile). Click on the Organization name, and you should be taken to the Azure DevOps Organization Home page.



4. Project dashboard



5. To manage user stories:

a. From the **left-hand navigation menu**, click on **Boards**. This will take you to the main **Boards** page, where you can manage work items, backlogs, and sprints.

b. On the **work items** page, you'll see the option to **Add a work item** at the top. Alternatively, you can find a **+** button or **Add New Work Item** depending on the view you're in. From the **Add a work item** dropdown, select **User Story**. This will open a form to enter details for the new User Story.

c.

Result:

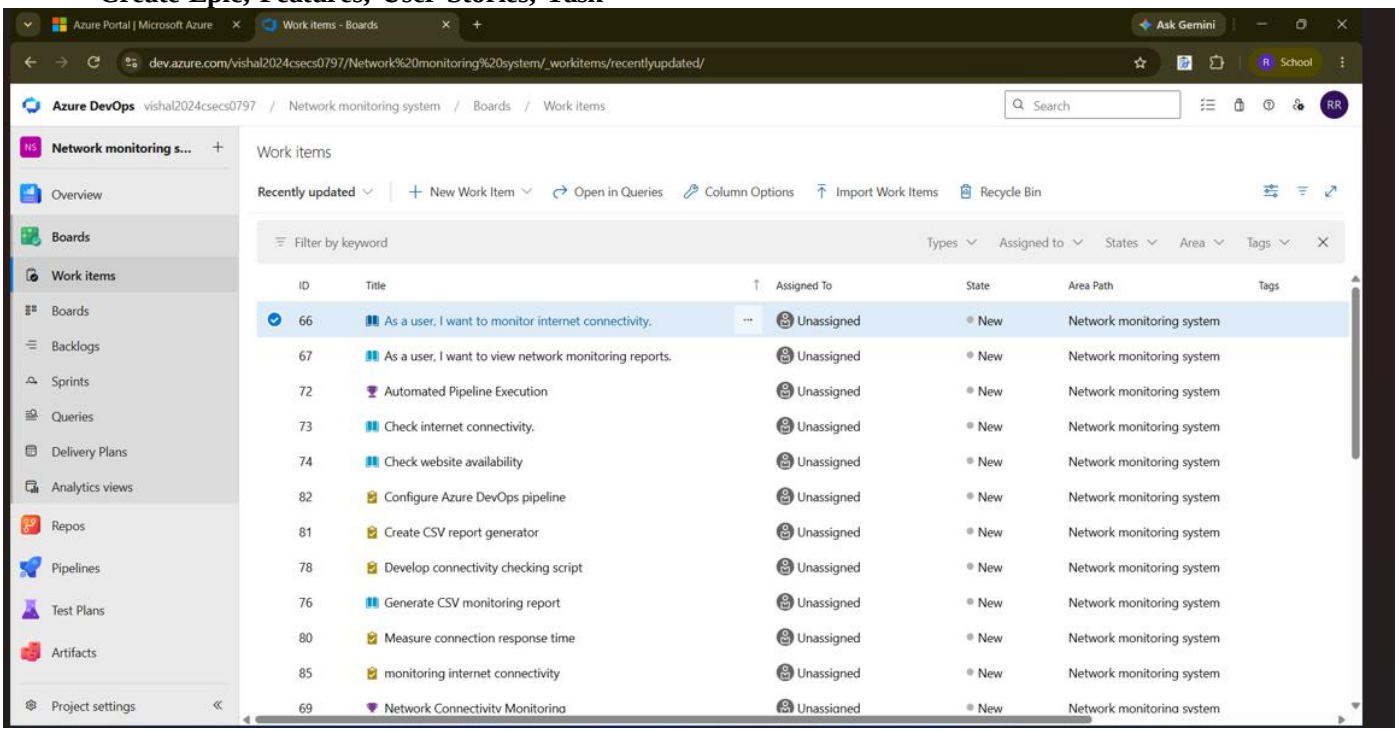
Successfully created an Azure DevOps project with user story management and agile workflow setup.

EXP NO: 3	SETTING UP EPICS, FEATURES, AND USER STORIES FOR PROJECT PLANNING

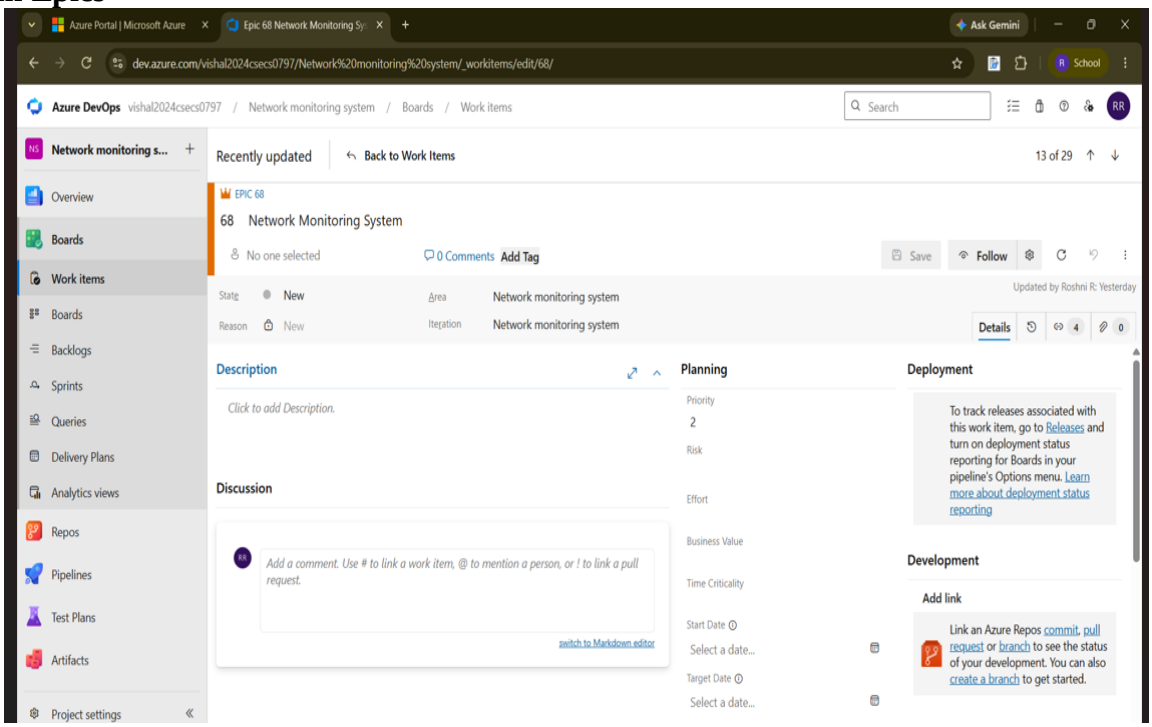
Aim:

To learn about how to create epics, user story, features, backlogs for your assigned project.

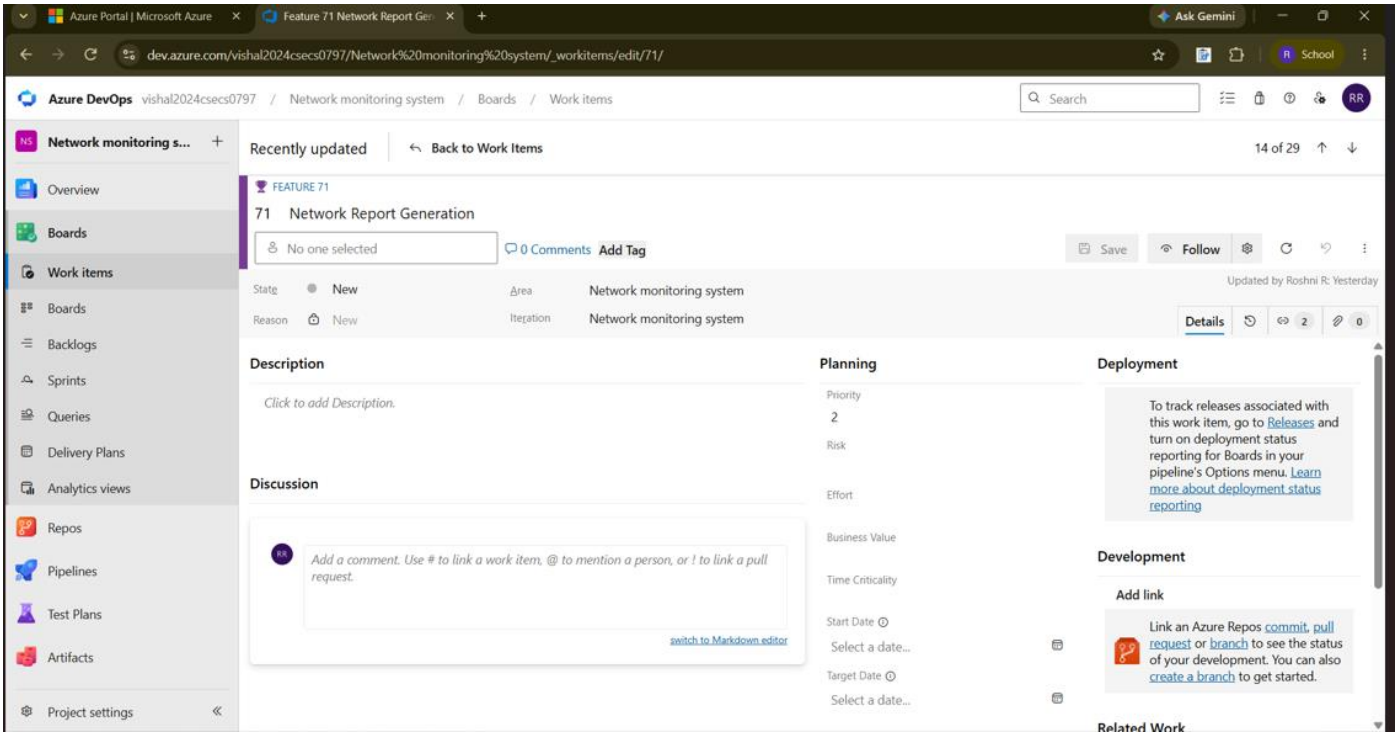
Create Epic, Features, User Stories, Task



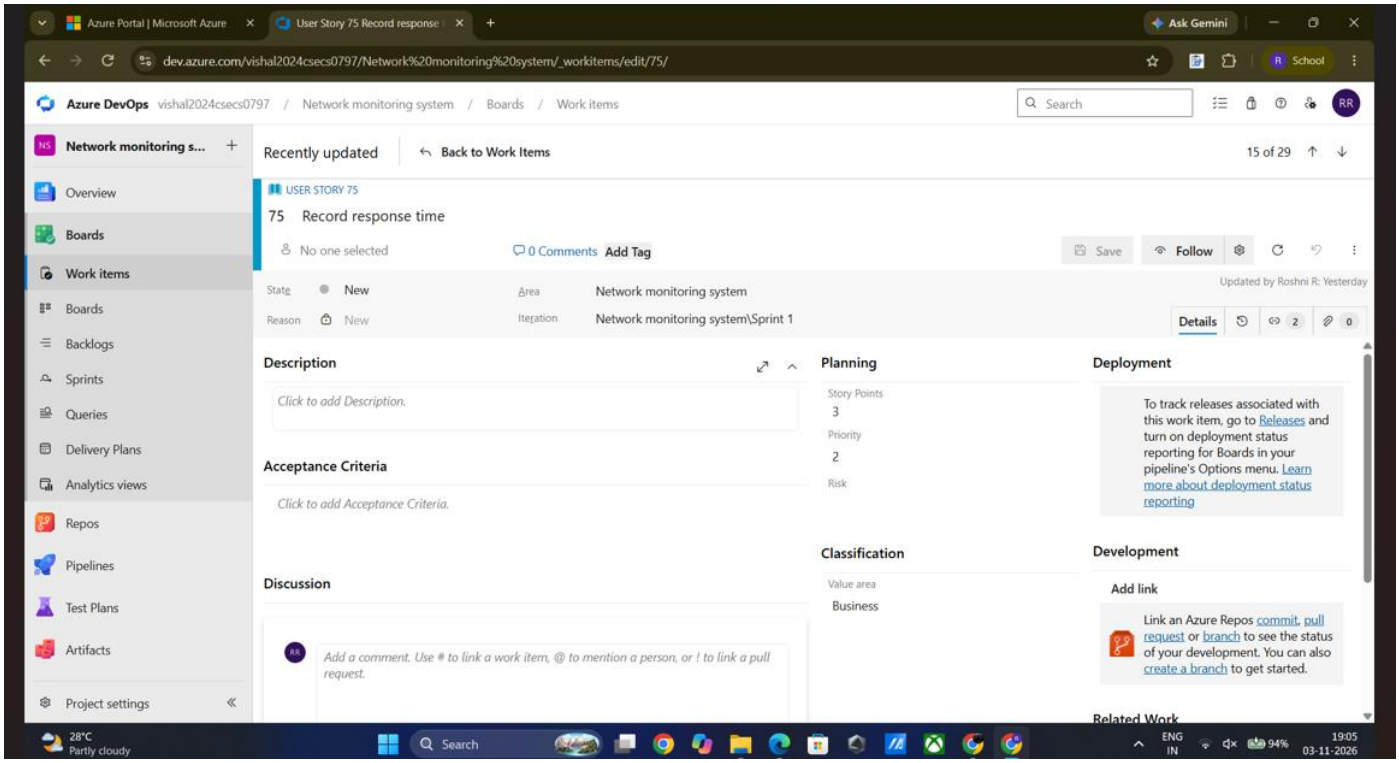
1. Fill in Epics



2. Fill in Features



3. Fill in User Story Details



Result:

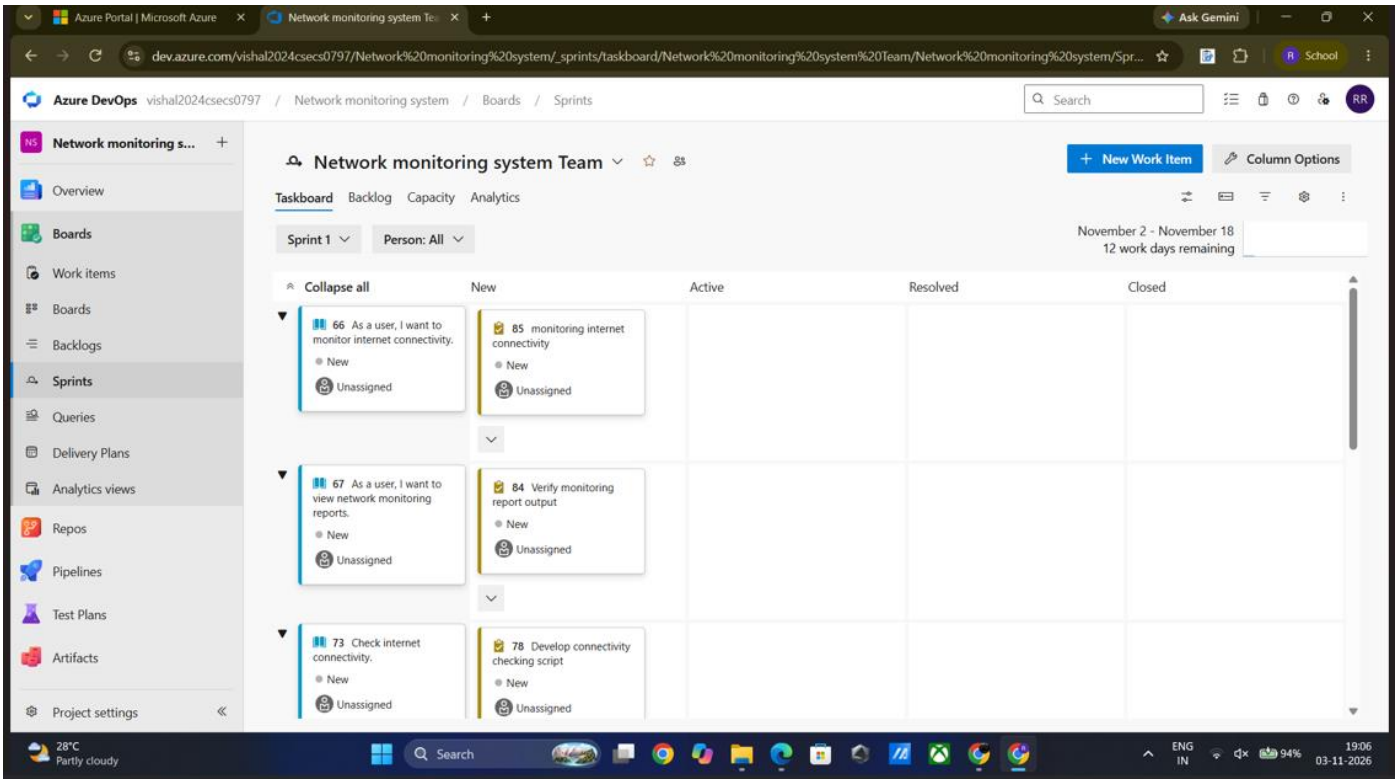
Thus, the creation of epics, features, user story and task has been created successfully.

EXP NO: 4	SPRINT PLANNING

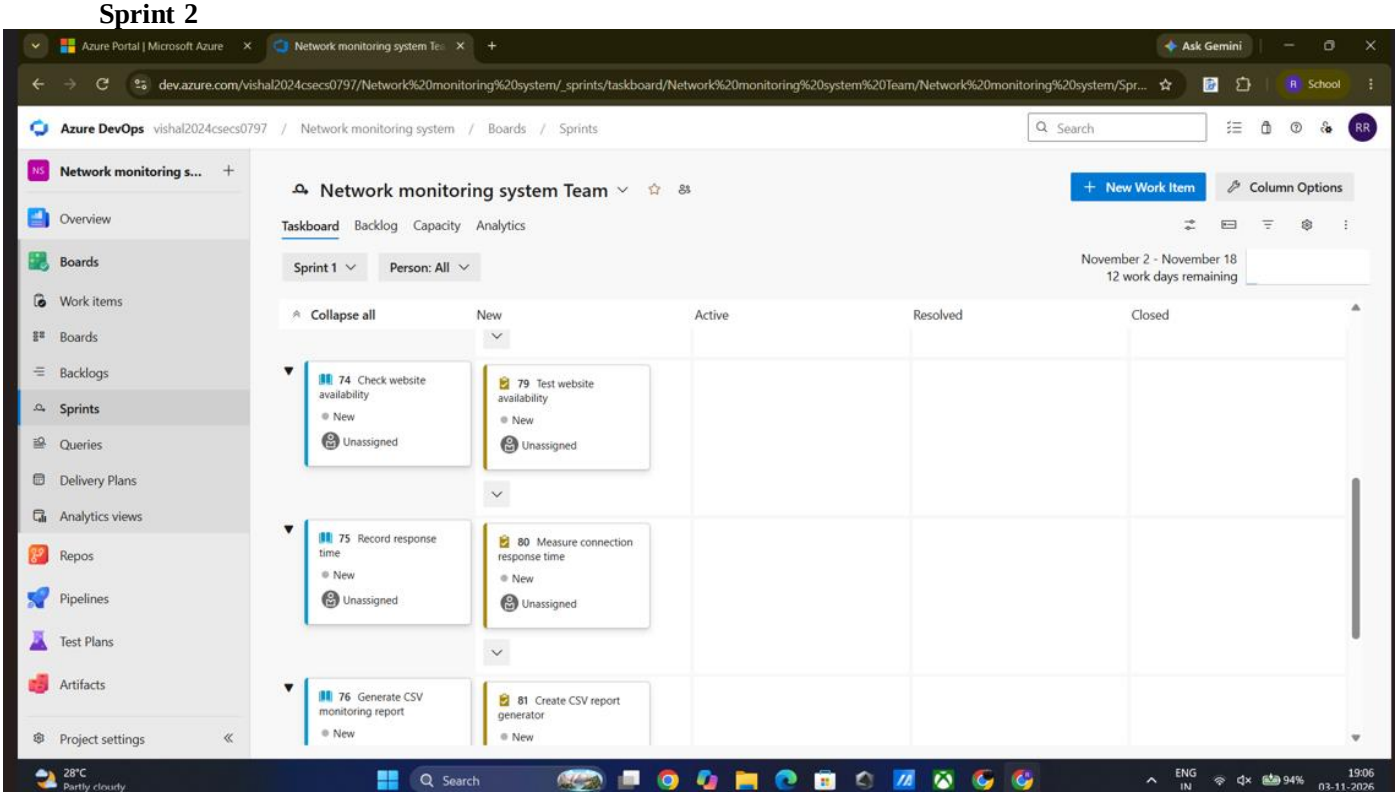
Aim:

To assign user story to specific sprint for the Network Monitoring System.

Sprint Planning
Sprint 1



Sprint 2


Azure Portal | Microsoft Azure
Network monitoring system Tea
Ask Gemini
dev.azure.com/vishal2024csecs0797/Network%20monitoring%20system/_sprints/taskboard/Network%20monitoring%20system%20Team/Network%20monitoring%20system/Spr...
School
Azure DevOps vishal2024csecs0797 / Network monitoring system / Boards / Sprints
Search
RR
Network monitoring s...
Overview
Boards
Work items
Boards
Backlogs
Sprints
Queries
Delivery Plans
Analytics views
Repos
Pipelines
Test Plans
Artifacts
Project settings
Network monitoring system Team
New Work Item
Column Options
Taskboard Backlog Capacity Analytics
Sprint 1
Person: All
November 2 - November 18
12 work days remaining
Collapse all
New
Active
Resolved
Closed
74 Check website availability
New
Unassigned
79 Test website availability
New
Unassigned
75 Record response time
New
Unassigned
80 Measure connection response time
New
Unassigned
76 Generate CSV monitoring report
New
81 Create CSV report generator
New
28°C
Partly cloudy
Search
ENG
IN
94%
19:06
03-11-2026

Result:

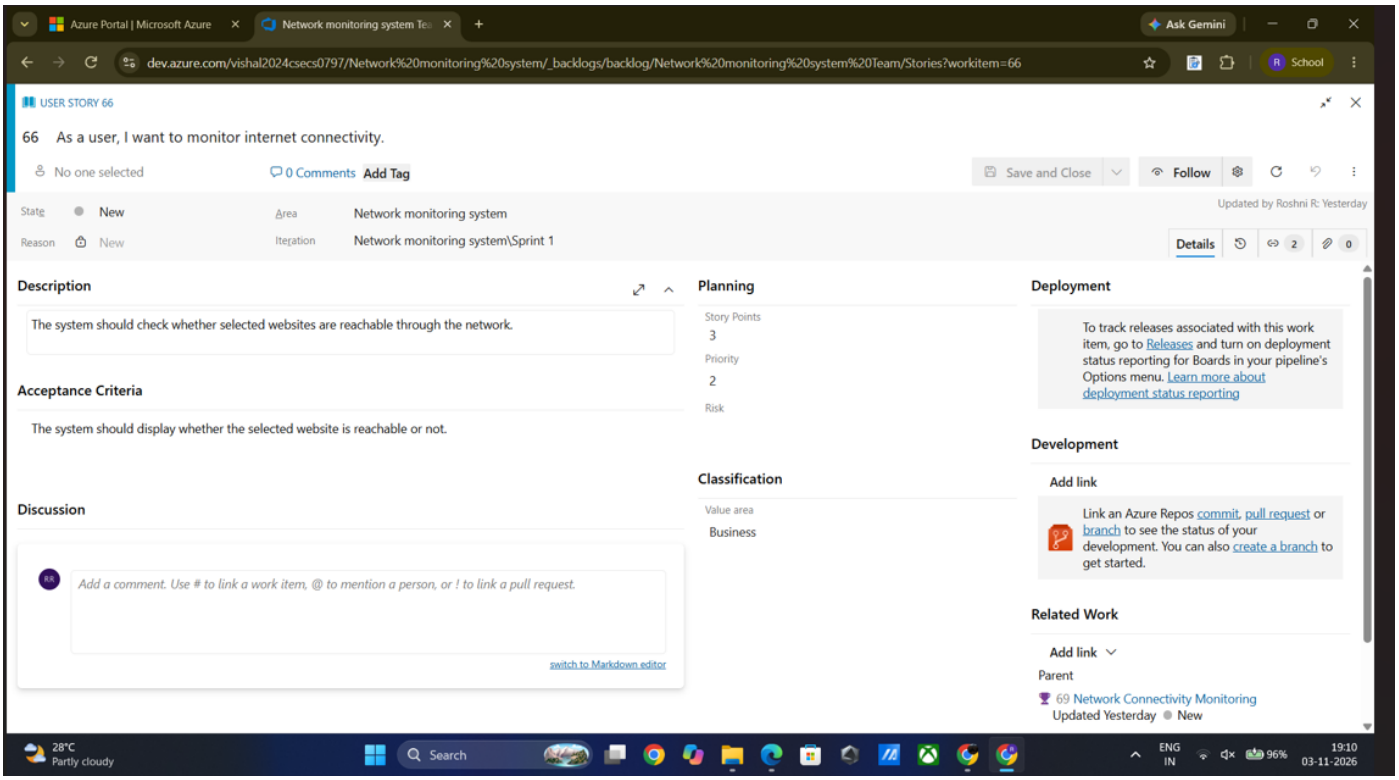
The Sprints are created for the Network Monitoring System Project

EXP NO: 5	POKER ESTIMATION

Aim:

Create Poker Estimation for the user stories – Network Monitoring System Project.

Poker Estimation



Result:

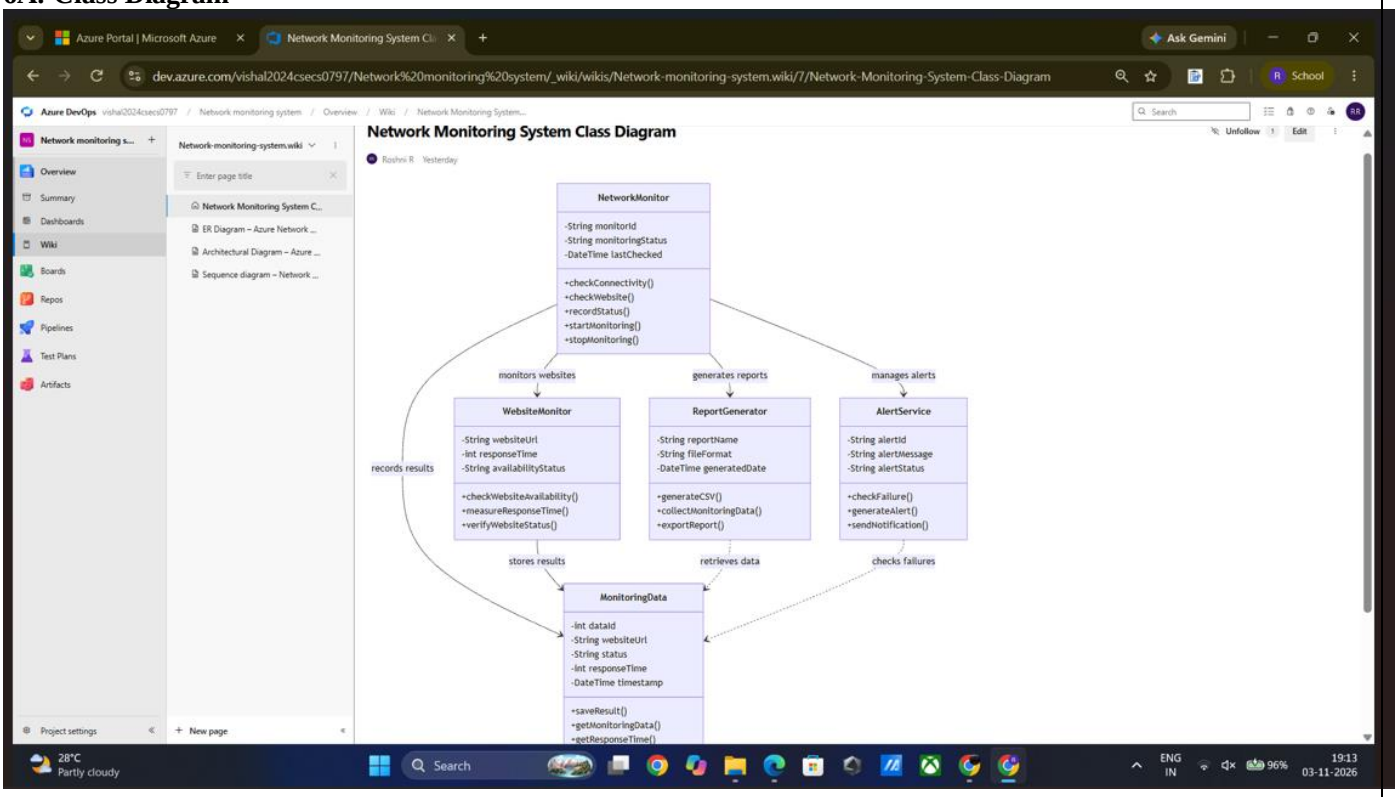
The Estimation/Story Points is created for the project using Poker Estimation.

EXP NO: 6	DESIGNING CLASS AND SEQUENCE DIAGRAMS FOR PROJECT ARCHITECTURE

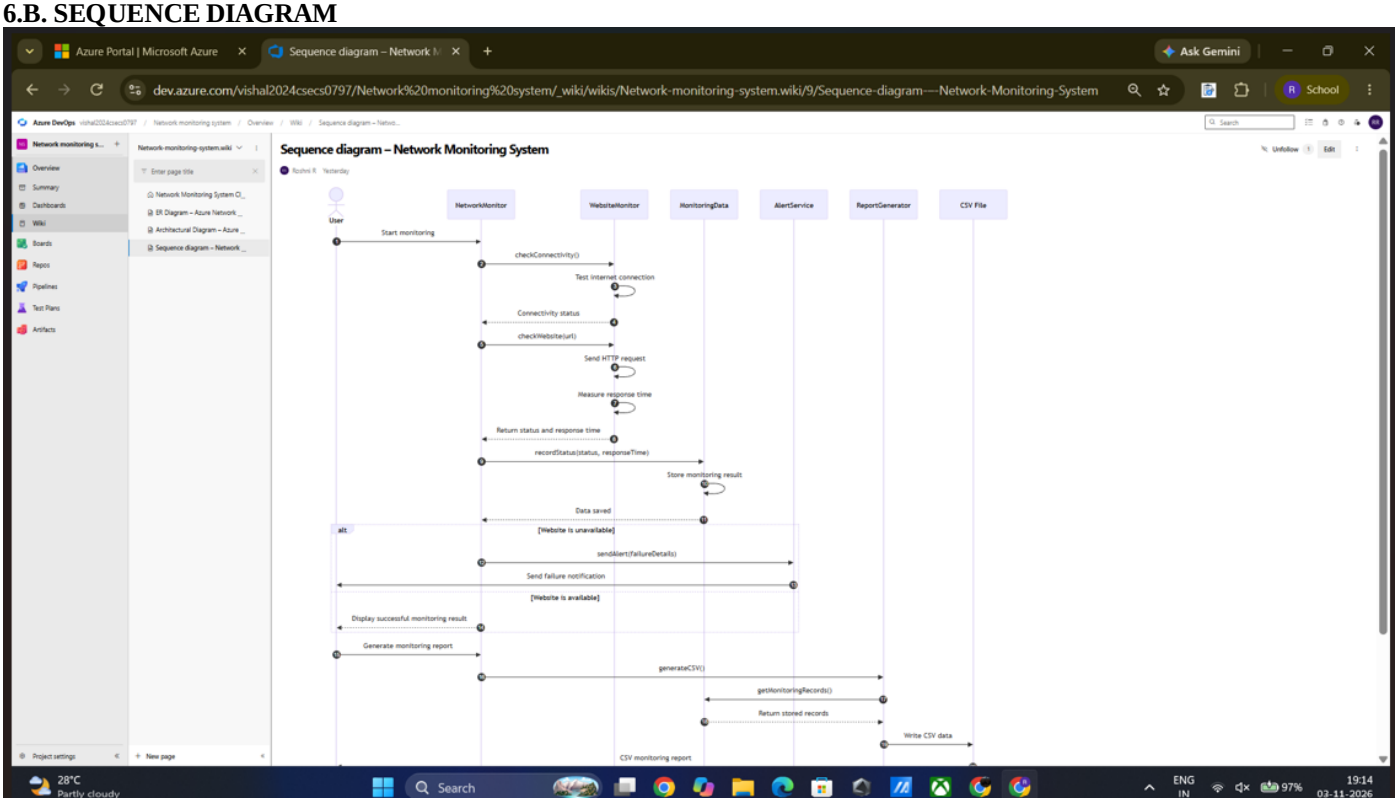
Aim:

To Design a Class Diagram and Sequence Diagram for the given Project.

6A. Class Diagram



6.B. SEQUENCE DIAGRAM



Result:

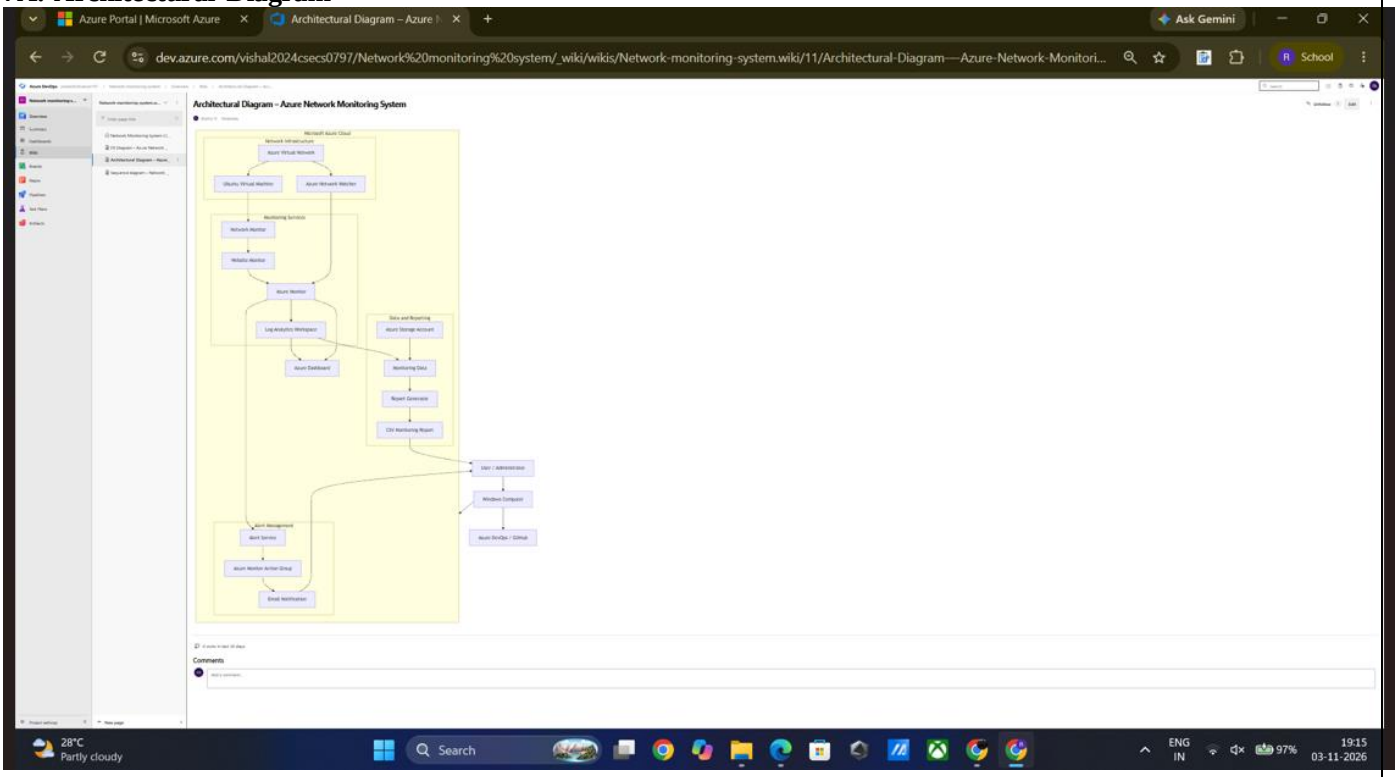
The Class Diagram and Sequence Diagram is designed Successfully for the Network Monitoring System.

EXP NO: 7	DESIGNING ARCHITECTURAL AND ER DIAGRAMS FOR PROJECT STRUCTURE

Aim:

To Design an Architectural Diagram and ER Diagram for the given Project.

7A. Architectural Diagram



Result:

The Architecture Diagram and ER Diagram is designed Successfully for the Network Monitoring System.

EXP NO: 8	TESTING – TEST PLANS AND TEST CASES

Aim:

Test Plans and Test Case and write two test cases for at least five user stories showcasing the happy path and error scenarios in azure DevOps platform.

Test Planning and Test Case Design Procedure – Network Monitoring System

1. Understand Core Features of the Application

- User Registration & Profile Management: New user sign-up, login, profile update, and password management.
- Search & Browse: Search for flights, hotels, tour packages, and activities based on dates, destination, filters (price, airline, star rating, etc.).
- Booking Management: Select options, add to cart, apply promo codes, proceed to checkout, and make payments.
- Booking History & Itinerary Management: View past and upcoming bookings, download e-tickets/vouchers, and cancel bookings (if allowed).
- Payment Gateway Integration: Secure transaction processing via credit/debit cards, net banking, and digital wallets.
- Admin Panel: Manage inventory (flights, hotels, tours), view all bookings, update pricing, and handle user inquiries.

2. Define User Interactions

Each test case will simulate real-world usage:

- Registering a new account and logging in.
- Searching for travel options with various filters.
- Selecting a flight/hotel/package and proceeding to book.
- Applying valid and invalid promo codes.
- Completing a payment transaction successfully and handling failures.
- Viewing and canceling a booking from the itinerary.
- An admin user managing flight seats and hotel room inventory.

3. Design Happy Path Test Cases

Focus on verifying that features work correctly under normal conditions:

- TC01 – User Registration with valid data
- TC02 – Login with valid credentials
- TC03 – Search and display available flights
- TC04 – Select a flight and proceed to booking page
- TC05 – Apply a valid promo code and verify discount
- TC06 – Complete a booking with successful payment
- TC07 – View booking confirmation and download e-ticket
- TC08 – Cancel a booking within the free cancellation period
- TC09 – Admin successfully adds a new flight to the inventory

4. Design Error Path Test Cases

Simulate incorrect inputs and unexpected behavior:

- TC10 – User Registration with an already registered email
- TC11 – Login with an incorrect password
- TC12 – Search for flights with no available routes (invalid date/route)
- TC13 – Attempt to book a flight without selecting seats/traveler details
- TC14 – Apply an invalid/expired promo code

- TC15 – Payment transaction failure due to insufficient funds
- TC16 – Attempt to cancel a non-refundable booking
- TC17 – Attempt to access admin panel with a standard user account

5. Break Down Steps and Expected Results

Test Case: TC06 – Complete a booking with successful payment

- **Steps:**
  1. Log in to the application.
  2. Search for a flight from "New York" to "London" for a future date.
  3. Select a flight from the results and click "Book Now".
  4. Fill in traveler details and proceed to payment.
  5. On the payment page, select "Credit Card", enter valid test card details.
  6. Click "Pay Now".
- **Expected Result:**
  - Payment is processed successfully.
  - A booking confirmation page is displayed with a PNR/Booking ID.
  - A confirmation email is sent to the registered email address.
  - The new booking appears in the "My Trips" section.

Test Case: TC14 – Apply an invalid/expired promo code

- **Steps:**
  1. Add a flight to the cart.
  2. On the checkout page, locate the promo code field.
  3. Enter an invalid code (e.g., "INVALID50") and click "Apply".
- **Expected Result:**
  - A clear error message is displayed: "The promo code you entered is invalid or has expired."
  - The total amount payable remains unchanged.

6. Use Clear Naming and IDs

Test Cases follow a consistent ID pattern:

- Format: TCXX – [Module] Description (e.g., TC03 – [Search] Search and display available flights)
- Example: TC07 – [Bookings] View booking confirmation and download e-ticket

This helps link cases directly to modules and features in the requirement traceability matrix.

7. Separate Test Suites

Organized by function for better manageability and assignment:

- User Authentication & Profile: Covers TC01, TC02, TC10, TC11.
- Search & Browsing: Covers TC03, TC12.
- Booking & Checkout Flow: Covers TC04, TC05, TC06, TC13, TC14, TC15.
- Itinerary & Post-Booking: Covers TC07, TC08, TC16.
- Admin & Backend Management: Covers TC09, TC17.

These suites can be directly mapped to Azure DevOps Test Plans for seamless test execution and bug tracking.

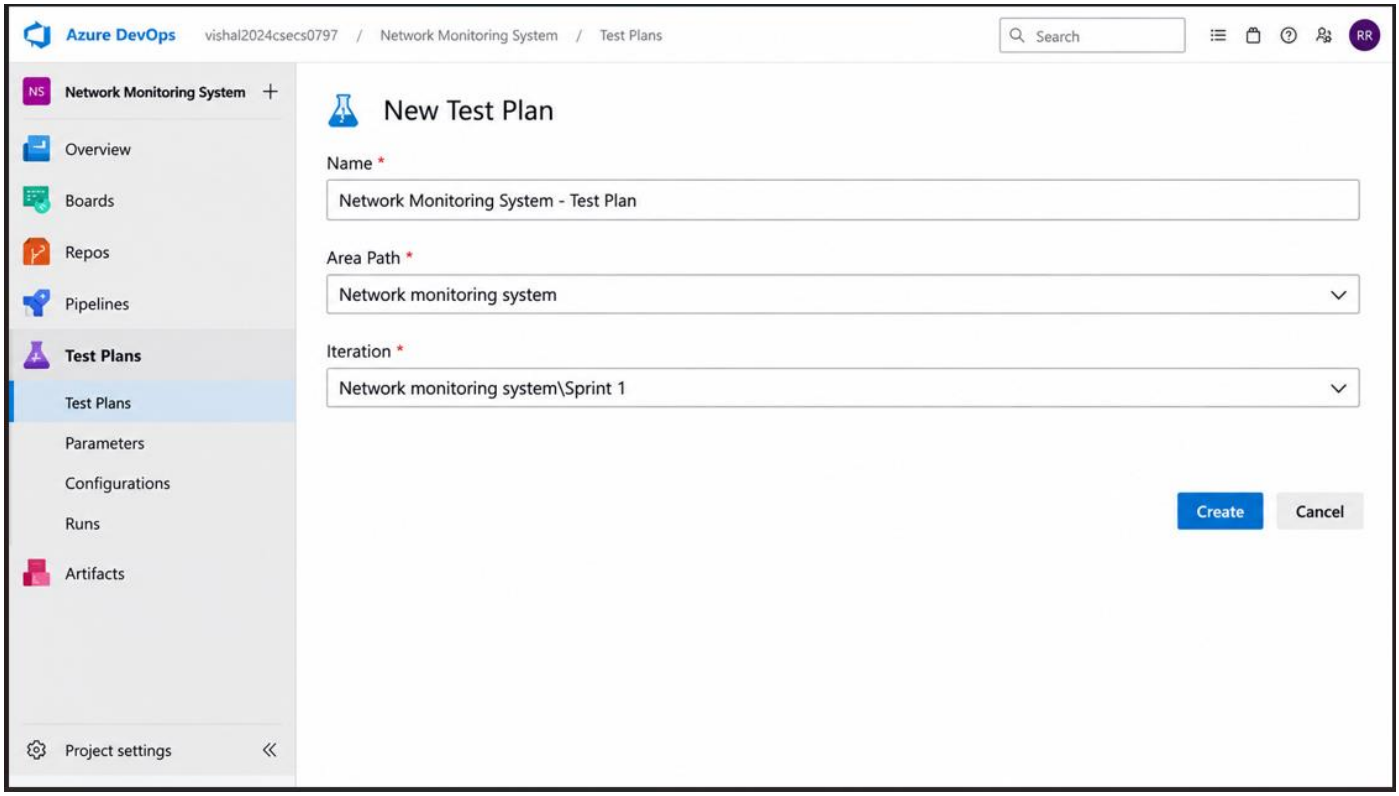
8. Prioritize and Review

- High Priority: Critical business flows like User Login (TC02), Flight Search (TC03), and Successful Payment (TC06).
- Medium Priority: Flows like Registration (TC01), Applying Promo Codes (TC05, TC14), and Viewing Itinerary (TC07).
- Low Priority: Edge cases and negative scenarios like searching for invalid routes (TC12).

All test cases are reviewed for:

- Feature Coverage: Ensuring all features from section 1 are tested.
- Traceability to Requirements: Each test case can be linked back to a specific user story or product requirement.
- Completeness and Accuracy: Steps are unambiguous, and expected results are clear and verifiable.

1. New test plan



2. Test case

Test Suite TS01 - User Authentication

User Story ID: US01
Title: As a user, I want to securely sign up and log in so that I can manage contracts

Test Case TC01 – Successful Sign Up
Type: Happy Path
Steps:

1. Navigate to the Sign-Up page
2. Enter valid name, email, and password
3. Click on Sign Up
   Expected Result: Account is created and user is redirected to dashboard

Test Case TC02 – Sign Up with Existing Email
Type: Error Path
Steps:

1. Navigate to the Sign-Up page
2. Enter a registered email
3. Click on Sign Up
   Expected Result: Error message is displayed saying email is already in use

Test Suite TS02 - Create Contract

User Story ID: US02
Title: As a user, I want to create a new contract with essential fields

Test Case TC03 – Create Contract Successfully
Type: Happy Path
Steps:

1. Log in
2. Click on Create Contract
3. Fill all required fields
4. Click Save
   Expected Result: Contract is created and visible in contract list

Test Case TC04 – Contract Creation with Missing Data
Type: Error Path
Steps:

1. Log in
2. Click on Create Contract
3. Leave mandatory fields blank

4. Click Save
   Expected Result: Validation error message shown and contract is not created

Test Suite TS03 - View Contract Details

User Story ID: US03
Title: As a user, I want to view contract details to verify status and content

Test Case TC05 – View Existing Contract
Type: Happy Path
Steps:

1. Log in

2. Go to Contracts list

3. Click on a contract
   Expected Result: Full contract details are displayed

Test Case TC06 – Contract Not Found
Type: Error Path
Steps:

1. Log in

2. Manually enter an invalid contract ID in the URL
   Expected Result: Message shown stating contract not found or does not exist

Test Suite TS04 - Approve or Reject Contract

User Story ID: US04
Title: As a user, I want to approve or reject a contract for review flow

Test Case TC07 – Approve Contract Successfully
Type: Happy Path
Steps:

1. Log in as reviewer

2. Open a pending contract

3. Click on Approve
   Expected Result: Status updates to Approved

Test Case TC08 – Try Approving Without Permission
Type: Error Path
Steps:

1. Log in as non-reviewer

2. Try to approve a contract
   Expected Result: Action denied with message stating insufficient permissions

Test Suite TS05 - Contract Expiry Alerts

User Story ID: US05
Title: As a user, I want to receive alerts before contract expiry

Test Case TC09 – Receive Expiry Alert
Type: Happy Path
Steps:

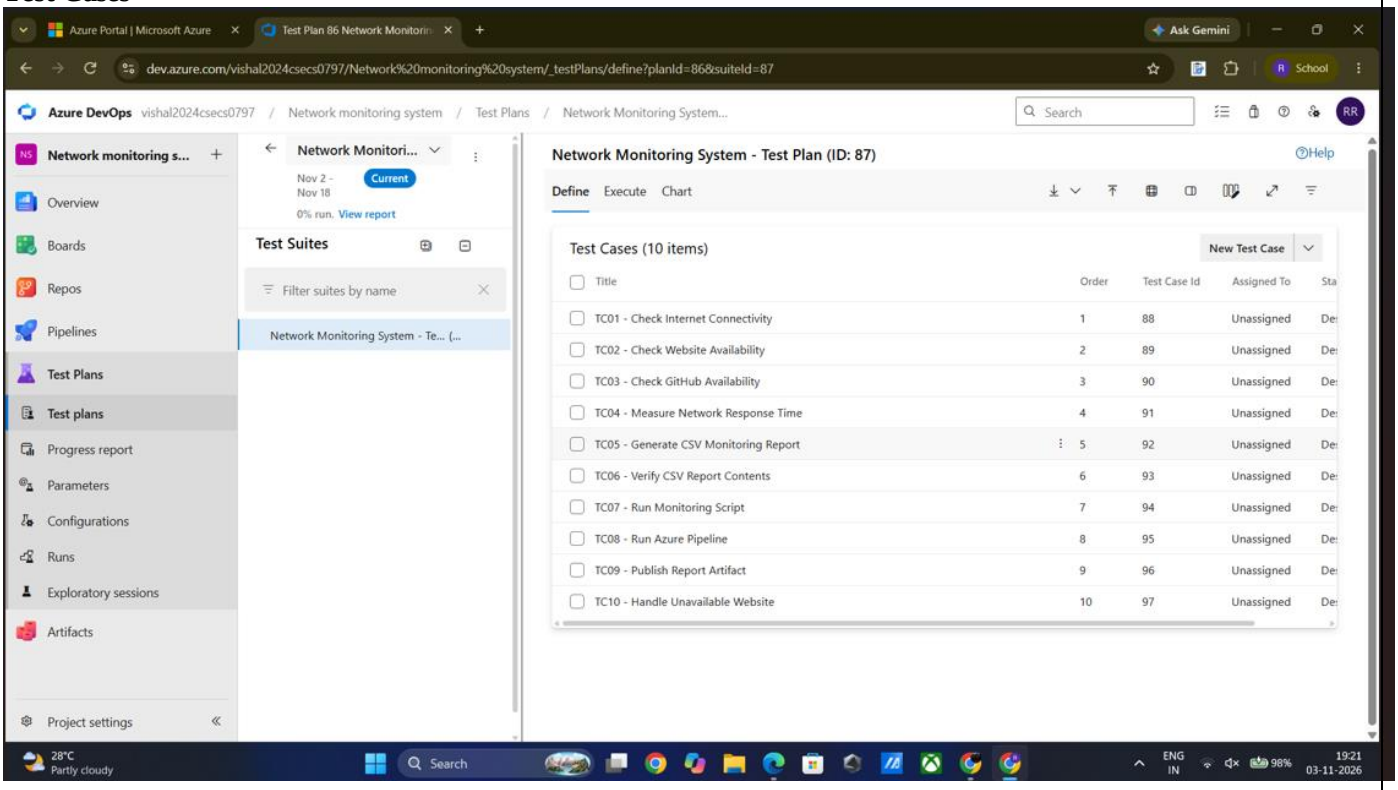
1. Log in

2. Check dashboard for contracts expiring within seven days
   Expected Result: Alert is shown for upcoming expiry

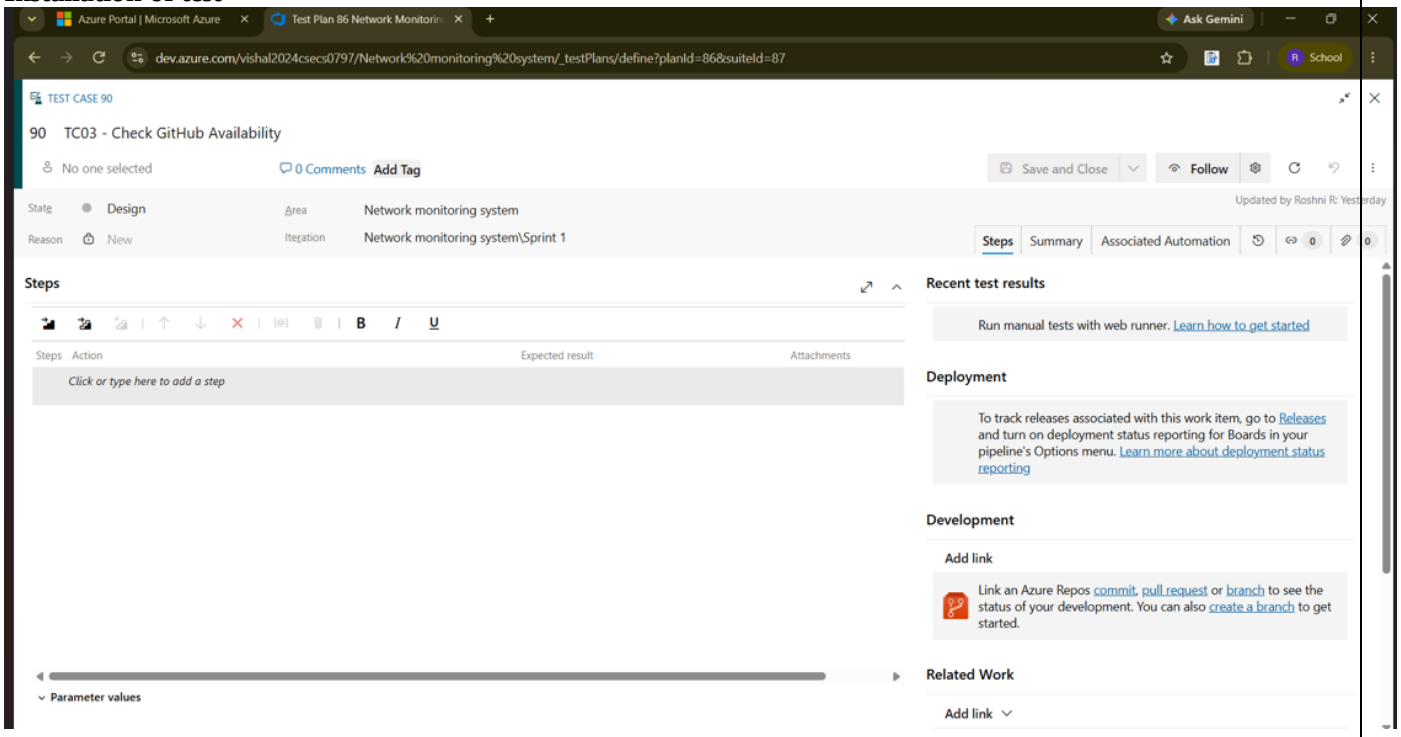
Test Case TC10 – No Alert for Expired Contracts
Type: Error Path
Steps:

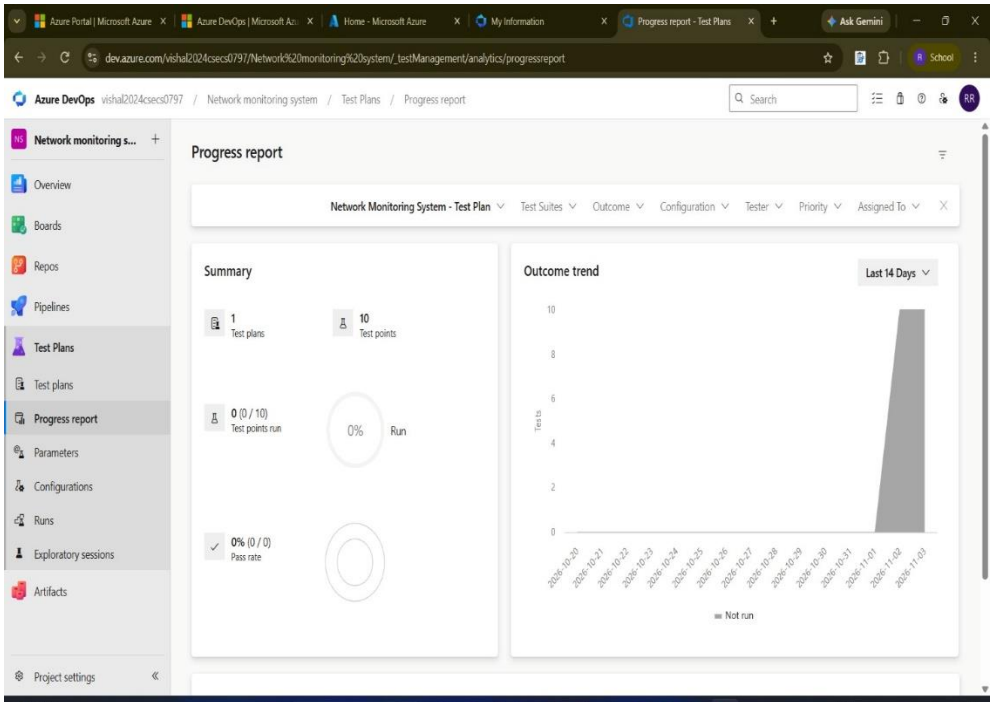
1. Log in after contract has already expired
   Expected Result: No alert shown and status shows expired

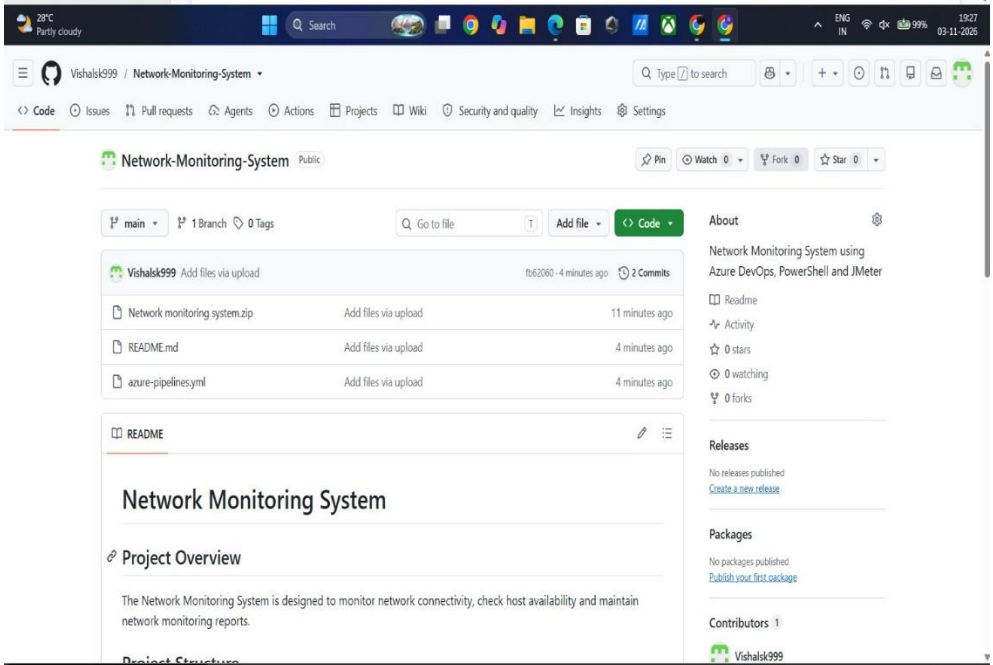
Test Cases

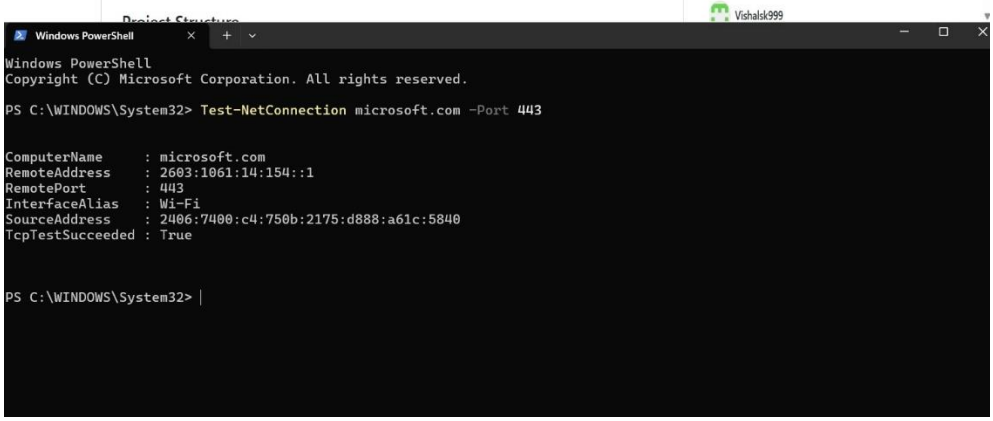


3. Installation of test




Azure DevOps
Network monitoring system
Test Plans
Progress report
Progress report
Network Monitoring System - Test Plan
Summary
1
Test plans
10
Test points
0 (0 / 10)
Test points run
0%
Run
0% (0 / 0)
Pass rate
Outcome trend
Last 14 Days
Not run


Vishalsk999 / Network-Monitoring-System
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
Network-Monitoring-System
Public
main
1 Branch
0 Tags
Add file
Code
About
Network Monitoring System using Azure DevOps, PowerShell and JMeter
Network monitoring system.zip
README.md
azure-pipelines.yml
Releases
Packages
Contributors
README
Network Monitoring System
Project Overview
The Network Monitoring System is designed to monitor network connectivity, check host availability and maintain network monitoring reports.


Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
PS C:\WINDOWS\System32> Test-NetConnection microsoft.com -Port 443
ComputerName : microsoft.com
RemoteAddress : 2603:1061:14:154::1
RemotePort : 443
InterfaceAlias : Wi-Fi
SourceAddress : 2406:7400:c4:750b:2175:d888:a61c:5840
TcpTestSucceeded : True
PS C:\WINDOWS\System32>

Result:

The test plans and test cases for the user stories is created in Azure DevOps with Happy Path and Error Path

EXP NO: 9	LOAD TESTING AND PIPELINES

Aim:

To create an Azure Load Testing resource and run a load test to evaluate the performance of a target endpoint and to create and demonstrate an Azure DevOps pipeline for automating application builds, tests, and deployment.

Load Testing

Azure Load Testing:
Azure Load Testing allows you to simulate high traffic and stress tests for your web applications and APIs to understand how they perform under load. It helps identify performance bottlenecks, scalability issues, and optimize resource usage before deployment.

Steps to Create an Azure Load Testing Resource:

Before you run your first test, you need to create the Azure Load Testing resource:

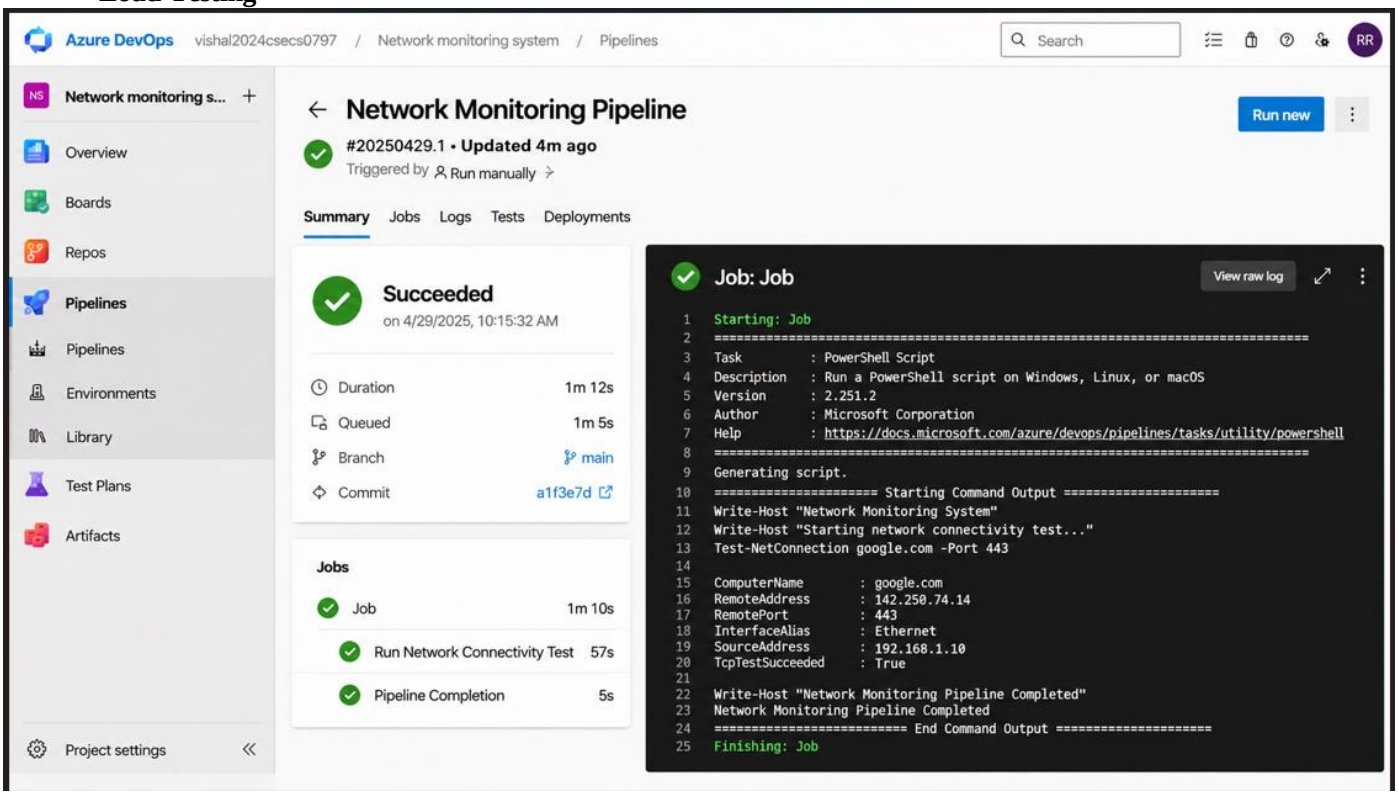
1. Sign in to Azure Portal
   Go to https://portal.azure.com and log in.
2. Create the Resource
   - Go to *Create a resource* → Search for "Azure Load Testing".
   - Select Azure Load Testing and click Create.
3. Fill in the Configuration Details
   - Subscription*:* Choose your Azure subscription.
   - *Resource Group:* Create new or select an existing one.
   - *Name:* Provide a unique name (no special characters).
   - *Location:* Choose the region for hosting the resource.
4. (Optional) Configure tags for categorization and billing.
5. Click Review + Create, then Create.
6. Once deployment is complete, click Go to resource.

Steps to Create and Run a Load Test:

Once your resource is ready:

1. Go to your Azure Load Testing resource and click Add HTTP requests > Create.
2. Basics Tab
   - *Test Name:* Provide a unique name.
   - *Description:* (Optional) Add test purpose.
   - *Run After Creation:* Keep checked.
3. Load Settings
   - *Test URL:* Enter the target endpoint (e.g., https://yourapi.com/products).
4. Click Review + Create → Create to start the test.

Load Testing



Pipelines

Description:

This experiment demonstrates how to connect a GitHub-hosted Flask-based music recommendation project with Azure DevOps. The pipeline will automatically install dependencies, run basic tests, and publish artifacts. This ensures that every commit triggers checks for reliability and smooth deployment.

Steps:

1. Connect GitHub to Azure DevOps:
    - In Azure DevOps, create a new project.
    - Create a pipeline and select GitHub as the source.
    - Authorize access to your GitHub repository, ensuring that Azure DevOps can pull the repository for your pipeline.
2. Create azure-pipelines.yml in Your Repo Root:
    - In your GitHub repository, create a new file called azure-pipelines.yml in the root directory.
    - Add the following basic pipeline configuration for Python and Flask:

yml Code

```
trigger:
  - main

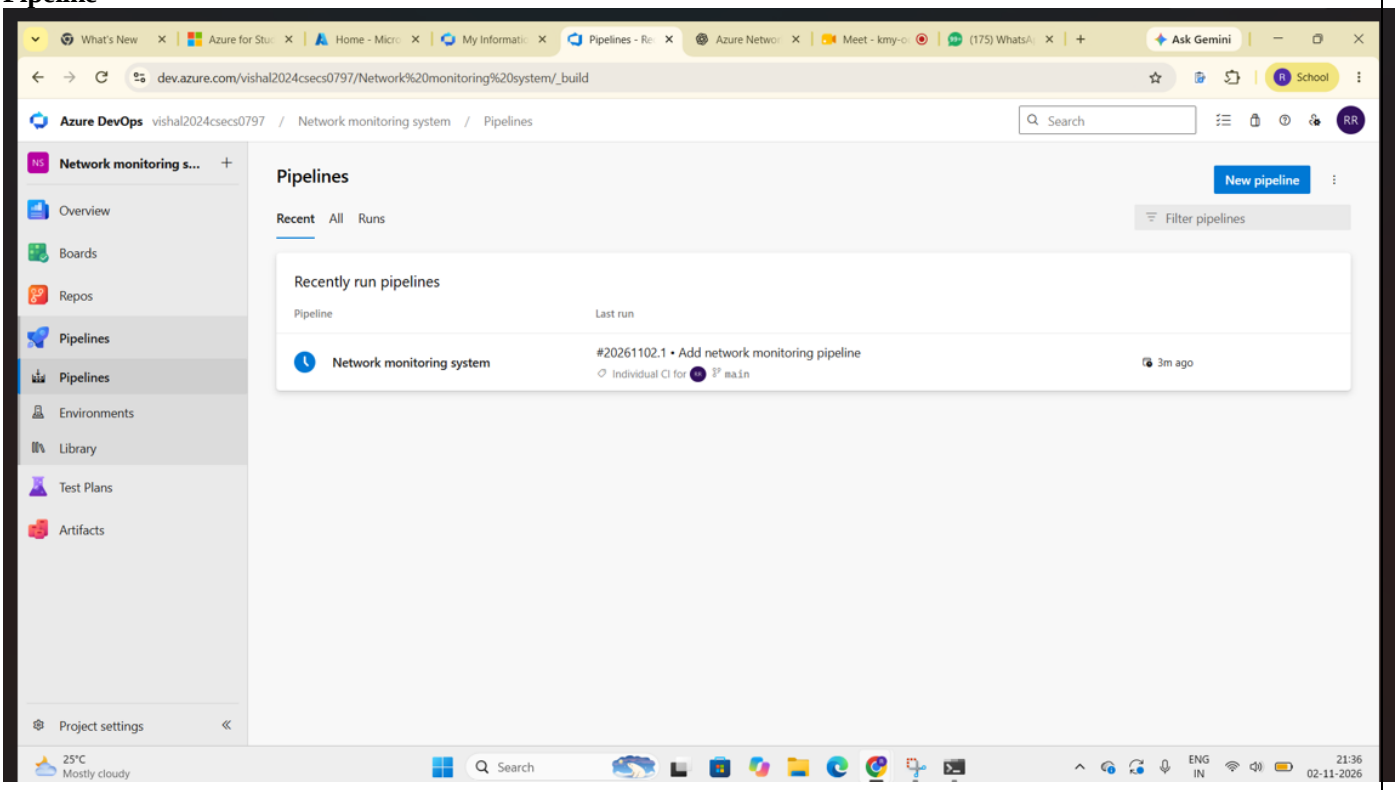
pool:
  vmImage: 'ubuntu-latest'

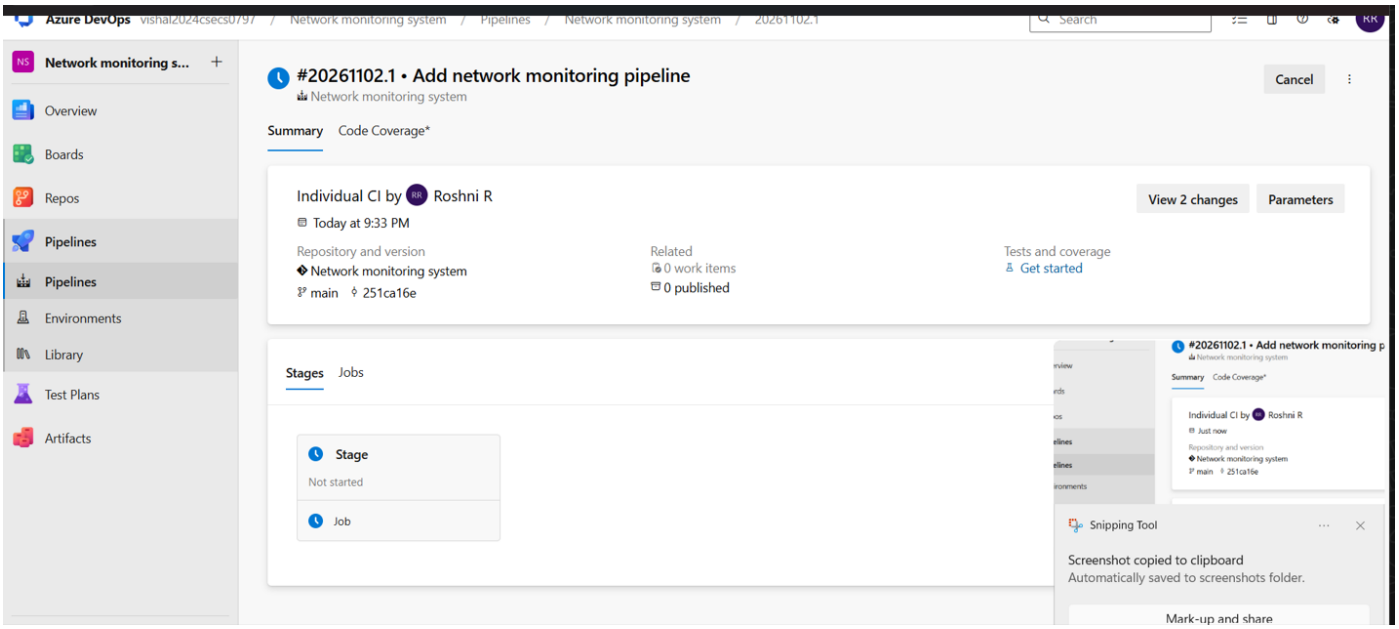
steps:
- checkout: self

- script: echo "■ Azure Pipeline for Network Monitoring System is running!"
                displayName: 'Run sample script'
```

3. Pipeline Tasks Include:
   - Setting up the Python environment using the UsePythonVersion task.
   - Installing project dependencies from project/requirements.txt. Make sure the path to requirements.txt is correct (it is located under the project folder).
   - Running a simple Python script to verify that Python is set up correctly and the pipeline works.
4. Run and Monitor Pipeline:
   - Commit changes to the main branch of your repository to trigger the pipeline in Azure DevOps.
   - Monitor the logs in the Azure DevOps portal to view logs, errors, or success messages and ensure everything runs smoothly.

Pipeline


Azure DevOps
vishal2024csecs0797 / Network monitoring system / Pipelines
Search
Network monitoring s...
Overview
Boards
Repos
Pipelines
Pipelines
Environments
Library
Test Plans
Artifacts
Project settings
Pipelines
New pipeline
Recent All Runs
Filter pipelines
Recently run pipelines
Pipeline
Last run
Network monitoring system
#20261102.1 • Add network monitoring pipeline
Individual CI for main
3m ago


Azure DevOps
Network monitoring s...
Overview
Boards
Repos
Pipelines
Pipelines
Environments
Library
Test Plans
Artifacts
#20261102.1 • Add network monitoring pipeline
Network monitoring system
Cancel
Summary
Code Coverage*
Individual CI by Roshni R
Today at 9:33 PM
View 2 changes
Parameters
Repository and version
Network monitoring system
main 251ca16e
Related
0 work items
0 published
Tests and coverage
Get started
Stages
Jobs
Stage
Not started
Job
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share

Result:

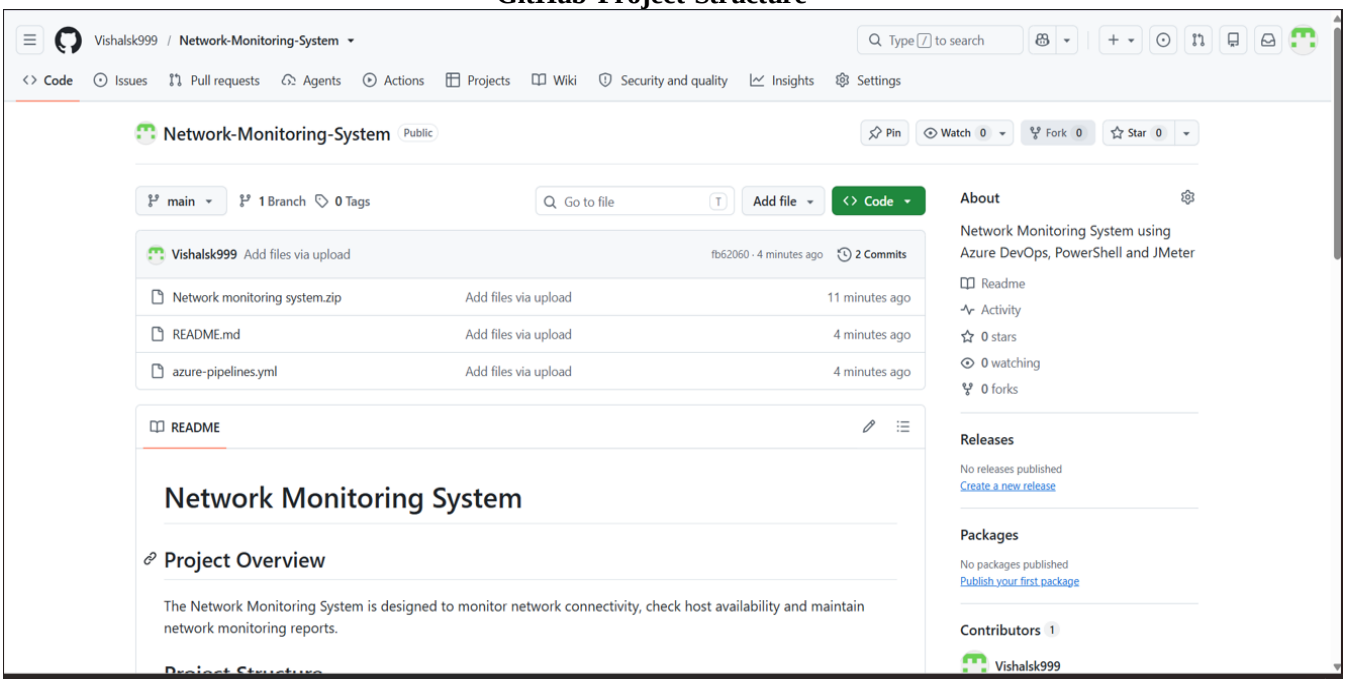
Successfully created the Azure Load Testing resource and executed a load test to assess the performance of the specified endpoint and also demonstrated pipelines in azure devops.

EXP NO: 10	GITHUB: PROJECT STRUCTURE & NAMING CONVENTIONS

Aim:

To provide a clear and organized view of the project's folder structure and file naming conventions, helping contributors and users easily understand, navigate, and extend the Network Monitoring System project.

GitHub Project Structure



Result:

The GitHub repository clearly displays the organized project structure and consistent naming conventions, making it easy for users and contributors to understand and navigate the codebase.